PROJECT REPORT ON

TINY RISC-V SYSTEM-ON-CHIP (SoC)

Design, Implementation and Functional Verification using Verilog/SystemVerilog HDL

Submitted in partial fulfilment of the requirements for the award of the degree

BACHELOR OF TECHNOLOGY

in

ELECTRONICS AND COMMUNICATION ENGINEERING

Prepared by

LEELA SHANMUKH YAGNEEK PATNALA

Department of Electronics and Communication Engineering

School of Electrical, Electronics and Communication Engineering

Vignan's Foundation for Science, Technology and Research (Deemed to be University)

Vadlamudi, Guntur, Andhra Pradesh – 522213, India

August – 2026

VIGNAN'S

FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH

(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

CERTIFICATE

This is to certify that the project entitled "Tiny RISC-V System-on-Chip (SoC) – Design, Implementation and Functional Verification" is an original RTL design project carried out by Leela Shanmukh Yagneek Patnala.

The project implements a 5-stage RISC-V RV32IM CPU subsystem with instruction/data cache, an AXI4-Lite interconnect, SDRAM controller, DMA controller, interrupt controller, and memory-mapped peripherals including GPIO, Timer, UART, SPI and I²C. Functional verification was performed using self-checking testbenches and simulation waveform analysis.

The work demonstrates practical knowledge of RTL design, processor architecture, memory-mapped communication, AXI4-Lite protocol implementation, cache integration, DMA-based data movement, interrupt handling, and functional verification.

Prepared By

LEELA SHANMUKH YAGNEEK PATNALA

DECLARATION

I hereby declare that the project entitled "**Tiny RISC-V System-on-Chip (SoC) – Design, Implementation and Functional Verification**" is an original work carried out as part of my self-learning and professional development in VLSI and RTL design.

The project has been designed and implemented using Verilog/SystemVerilog HDL with the objective of understanding **RISC-V processor** architecture, pipelined CPU operation, cache organization, **AXI4-Lite** communication, memory-mapped peripherals, **DMA** transfers, **interrupt handling**, and functional verification.

The work presented in this report is based on the implemented project and its simulation results. I take responsibility for the design, implementation, verification, and documentation of the project.

Place: Tenali-Andhra pradesh
Date: 12-08-2026

Signature

LEELA SHANMUKH YAGNEEK PATNALA

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who contributed directly or indirectly to the successful completion of this **Tiny RISC-V SoC** project.

This project provided valuable hands-on experience in **RISC-V processor** design, five-stage pipeline implementation, cache architecture, **AMBA AXI4-Lite** interfacing, memory systems, peripheral integration, **DMA** data transfers, interrupt controllers, and RTL verification.

I am thankful to the open-source hardware community, technical documentation, simulation tools, and educational resources that supported my understanding of processor and SoC design.

I also extend my gratitude to my mentors, faculty members, peers, and everyone whose guidance and discussions supported the development and verification of this work.

Prepared By
LEELA SHANMUKH YAGNEEK PATNALA

B.Tech – Electronics and Communication Engineering

ABSTRACT

The Tiny RISC-V System-on-Chip (SoC) is a compact RTL-based processor system designed around a 5-stage RISC-V RV32IM CPU. The architecture combines a pipelined CPU, instruction and data cache, AXI4-Lite interconnect, SDRAM controller, DMA controller, interrupt controller, and multiple memory-mapped peripherals in a modular SoC structure.

The CPU uses the five pipeline stages Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). A 2-way set-associative cache subsystem provides instruction and data caching. The cache interface operates as an AXI4-Lite master and connects the processor subsystem to the AXI4-Lite interconnect.

The AXI4-Lite interconnect contains independent read and write paths and an address decoder that routes transactions to memory and peripherals. The implemented peripheral address map includes GPIO, Timer, UART, SPI, I²C, DMA and Interrupt Controller regions. The DMA controller supports memory-to-memory and memory-to-peripheral style transfers, configurable transfer size, and completion interrupt generation.

The complete design was verified using self-checking Verilog/SystemVerilog testbenches and simulation waveform analysis. The available verification results show successful CPU bring-up, AXI4-Lite read/write handshaking, SDRAM operation, GPIO, UART, SPI, I²C, DMA, interrupt-controller verification, and DMA-cache integration. The final SoC verification report records 10/10 tests passed.

Keywords: RISC-V, RV32IM, CPU, 5-stage pipeline, cache, AXI4-Lite, SDRAM, GPIO, Timer, UART, SPI, I²C, DMA, Interrupt Controller, RTL, Verilog, SystemVerilog, SoC, Functional Verification.

CONTENTS

CHAPTER 1: INTRODUCTION

1.1 Introduction

A System-on-Chip integrates processor, memory, interconnect, and peripheral functions into a single digital system. This project develops a Tiny SoC around a RISC-V processor and focuses on the RTL implementation and functional verification of the complete subsystem.

The design combines a 5-stage RV32IM CPU with a cache subsystem and an AXI4-Lite interconnect. The interconnect provides a structured memory-mapped path to external memory and peripheral slaves, while DMA and interrupt hardware provide mechanisms for efficient data movement and event handling.

1.2 Objectives

- Implement a 5-stage RISC-V RV32IM CPU subsystem.
- Integrate instruction and data caching using a 2-way set-associative cache architecture.
- Implement AXI4-Lite master/slave communication and independent read/write transaction paths.
- Develop an address decoder for memory-mapped RAM and peripherals.
- Integrate SDRAM controller, GPIO, Timer, UART, SPI, I²C, DMA and Interrupt Controller blocks.
- Implement DMA data transfers and completion signaling.
- Verify individual modules and the integrated SoC using self-checking simulation testbenches.
- Analyze waveforms to confirm AXI handshakes, peripheral accesses, DMA activity and interrupts.

1.3 Problem Statement

A compact processor system requires reliable communication between the CPU, memory and peripherals. The design challenge is to integrate these blocks while maintaining correct address decoding, bus handshaking, data movement, reset behavior and interrupt signaling. The project addresses this challenge through a modular RISC-V SoC architecture using AXI4-Lite and structured RTL verification.

1.4 Scope

The scope covers RTL architecture, module integration, simulation-based functional verification, cache and DMA integration, and waveform analysis. The current project evidence is simulation-based; hardware-specific results should only be added when FPGA validation is actually performed.

CHAPTER 2: PROPOSED SYSTEM / ARCHITECTURE

2.1 System Overview

The Tiny SoC is organized into a CPU subsystem, memory system, AXI4-Lite interconnect, DMA and interrupt subsystems, and AXI4-Lite peripheral slaves.

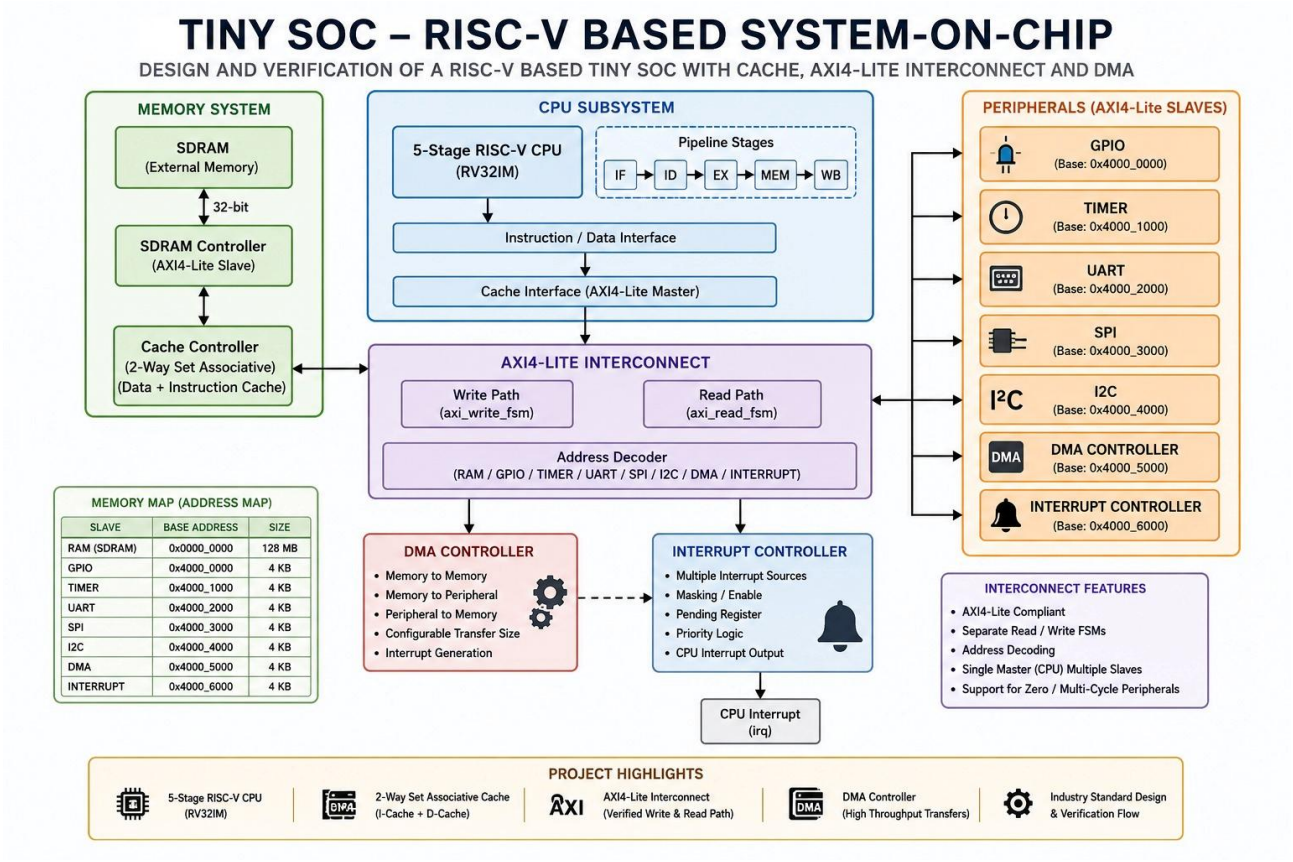


Figure 2.1: Complete Tiny RISC-V SoC architecture .

2.2 CPU Subsystem

The CPU subsystem contains a 5-stage RISC-V RV32IM processor. The pipeline is divided into Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). The instruction/data interface connects the CPU to the cache subsystem.

Pipeline Stage	**Function**
IF	Fetch instruction using the program counter.
ID	Decode instruction and read source registers.
EX	Perform ALU operation, branch calculation and address generation.
MEM	Perform load/store memory access.
WB	Write results back to the register file.

2.3 Cache Subsystem

The design uses a 2-way set-associative cache organization with instruction and data cache functionality. The cache controller interfaces with the CPU on one side and acts as an AXI4-Lite master toward the interconnect. This reduces repeated memory accesses and separates processor-side access from the external memory path.

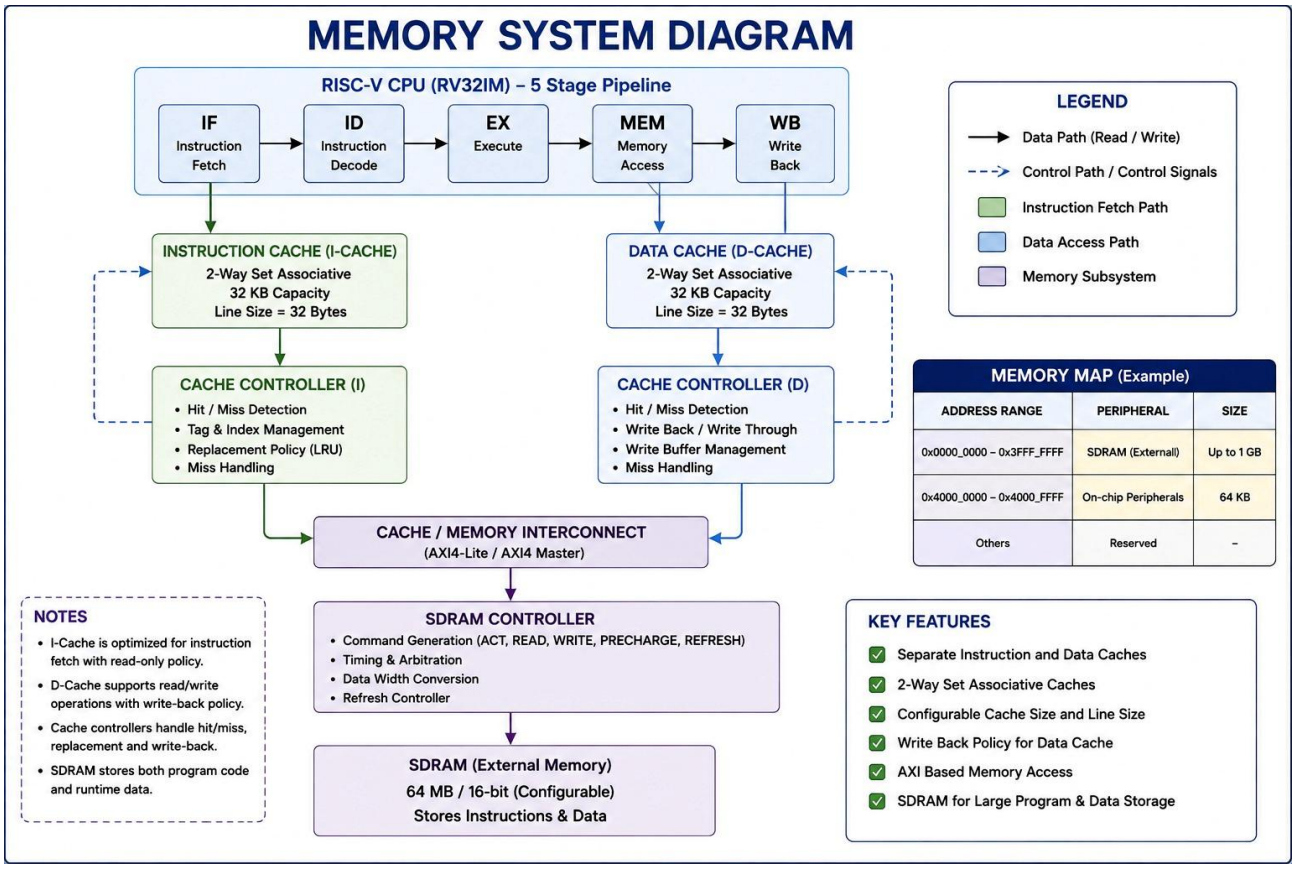


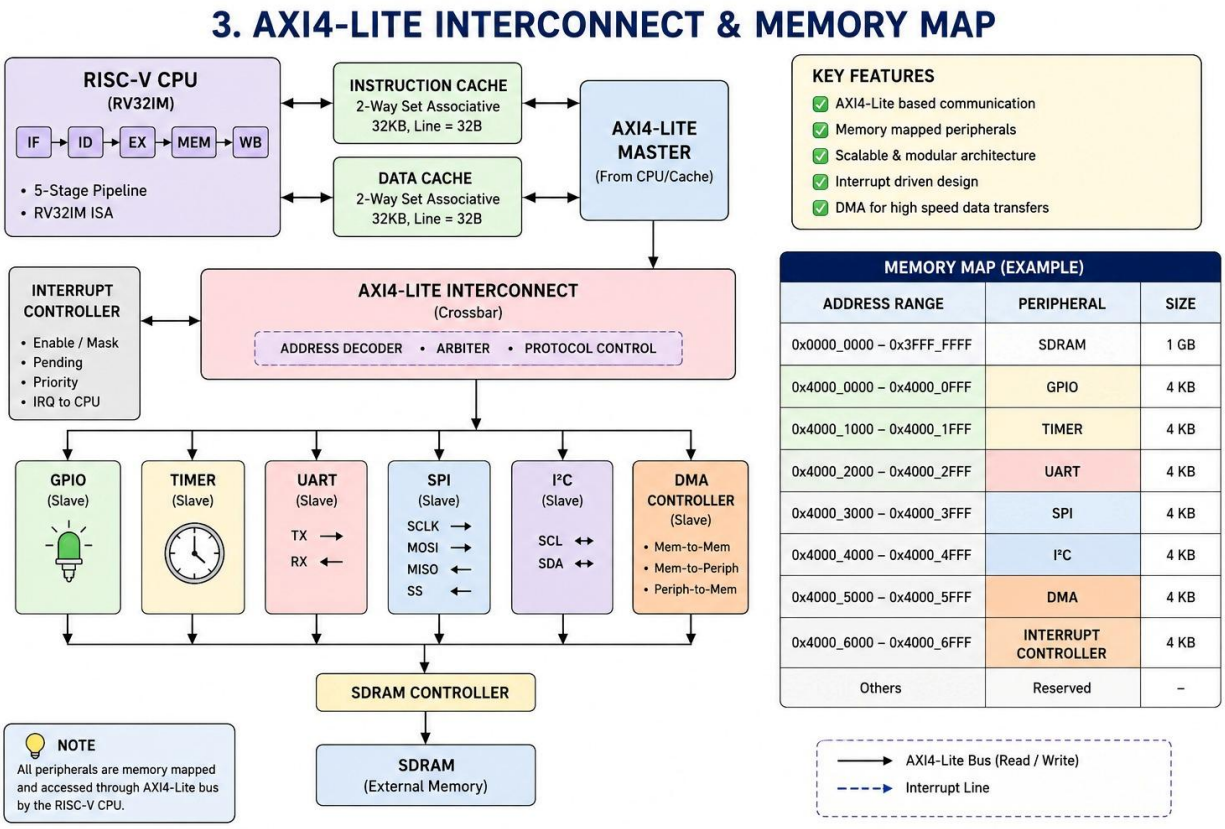
Figure 2.2: Memory system showing cache controller and SDRAM path.

2.4 AXI4-Lite Interconnect

The AXI4-Lite interconnect provides the communication fabric between the CPU/cache master and the memory/peripheral slaves. Separate read and write finite-state-machine paths manage transactions. The address decoder determines the selected slave from the transaction address.

2.5 AXI Read/Write Handshaking

AXI4-Lite uses independent VALID/READY handshakes. For a write, the write address and write data channels are accepted using AWVALID/AWREADY and WVALID/WREADY, followed by a write response. For a read, the address channel uses ARVALID/ARREADY and the slave returns data and response through the read response channel.



2.6 Memory System

The memory system includes an SDRAM controller and external SDRAM. The memory map assigns the RAM/SDRAM region starting at 0x0000_0000 with a documented size of 128 MB. The SDRAM controller provides the AXI4-Lite slave-side memory access used by the SoC.

2.7 DMA Controller

The DMA controller transfers data without requiring the CPU to perform every individual data movement operation. The implemented design supports memory-to-memory and memory-to-peripheral style transfers, configurable transfer size, and interrupt generation on completion.

2.8 Interrupt Controller

The interrupt controller collects interrupt sources, provides masking/enabling and pending status, applies priority logic, and produces the CPU interrupt request (irq). This allows peripherals and DMA to signal events to the processor.

CHAPTER 3: WORKING PRINCIPLE

3.1 CPU Operation

1. The program counter selects the next instruction.
2. The instruction is decoded and source registers are read.
3. The execute stage performs arithmetic, logical, branch or address-generation operations.
4. Memory operations are routed through the instruction/data interface and cache subsystem.
5. Results are written back to the register file in the WB stage.

3.2 Memory-Mapped Access Flow

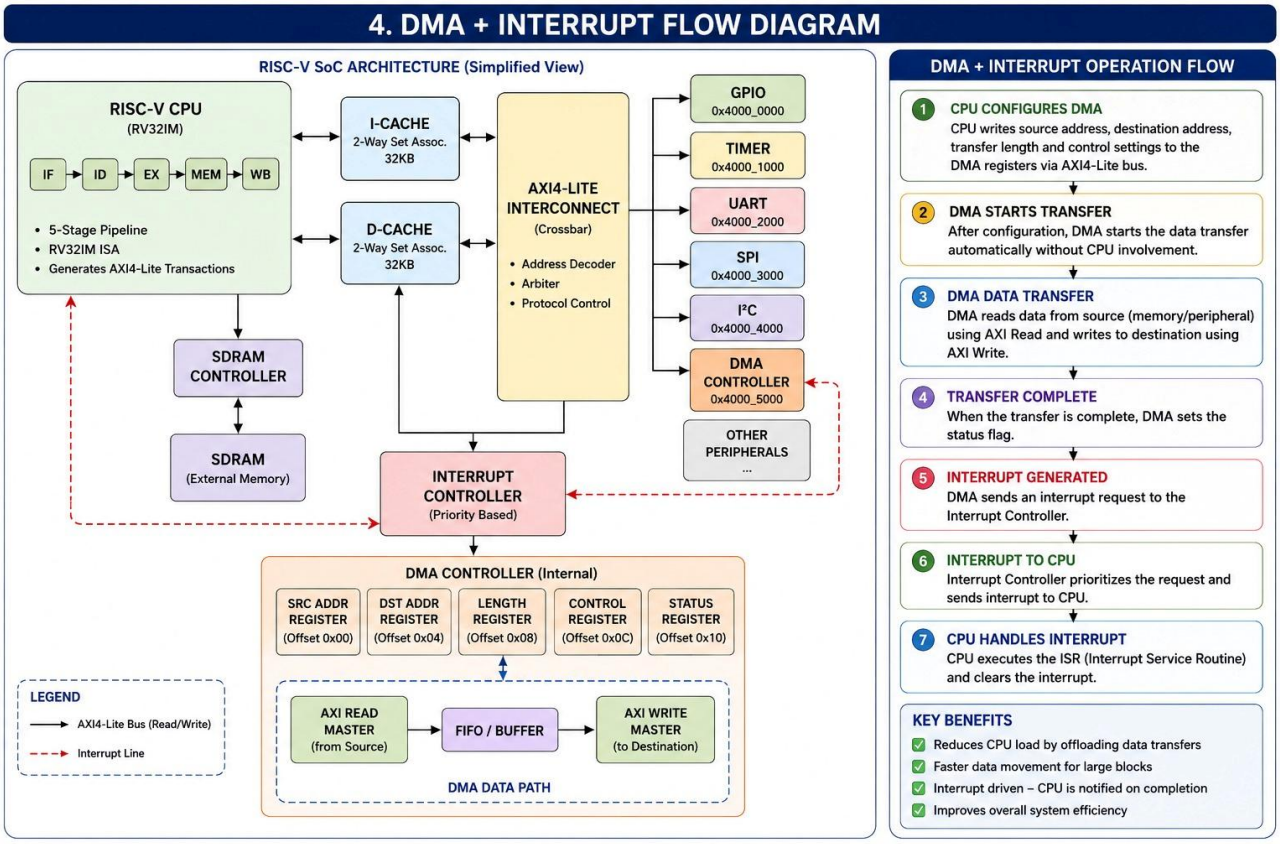
6. CPU/cache generates an address and read/write request.
7. The AXI4-Lite master presents the transaction on the appropriate AXI channel.
8. The interconnect performs address decoding.
9. The selected slave receives the request.
10. The slave returns read data or a write response.
11. The response is propagated back to the CPU/cache.

3.3 Peripheral Address Map

Slave	**Base Address**	**Size**
RAM (SDRAM)	0x0000_0000	128 MB
GPIO	0x4000_0000	4 KB
Timer	0x4000_1000	4 KB
UART	0x4000_2000	4 KB
SPI	0x4000_3000	4 KB
I²C	0x4000_4000	4 KB
DMA	0x4000_5000	4 KB
Interrupt Controller	0x4000_6000	4 KB

3.4 DMA Operation Flow

12. CPU programs the DMA source, destination and transfer parameters.
13. DMA starts the requested transfer.
14. DMA performs memory read/write activity through the system memory path.
15. Transfer count is updated as data moves.
16. DMA asserts completion when the requested transfer is finished.
17. The completion event can generate an interrupt.



3.5 Interrupt Operation

When a peripheral or DMA generates an interrupt source, the interrupt controller records the pending event, applies its enable/mask and priority logic, and asserts the CPU irq output when an enabled interrupt is serviced.

CHAPTER 4: IMPLEMENTATION AND SPECIFICATIONS

4.1 Design Specifications

Parameter	Specification
CPU	5-stage RISC-V RV32IM
Pipeline	IF / ID / EX / MEM / WB
Cache	2-way set-associative; instruction + data cache
Interconnect	AXI4-Lite
Interconnect structure	Separate read/write FSMs with address decoder
Memory	External SDRAM through SDRAM controller
Data width	32-bit documented memory data path
DMA	Memory-to-memory and memory-to-peripheral style transfers
Interrupts	Multiple sources with masking, pending and priority logic
Peripherals	GPIO, Timer, UART, SPI, I²C, DMA, Interrupt Controller
Verification	Self-checking simulation testbenches + waveform analysis

4.2 RTL Module Organization

The project is organized into modular RTL source files. The observed implementation includes CPU, pipeline registers, AXI master, AXI interconnect, peripheral block, SDRAM, cache package/controller, SPI master, I²C master, interrupt controller and DMA controller modules, together with package/header files and memory initialization files.

4.3 Software / Instruction Test

The CPU bring-up test uses RISC-V instructions loaded from program memory. The reported verification result shows x1 = 0x00000005, x2 = 0x0000000A, x3 = 0x0000000F and x4 = 0x0000000F, with final PC = 0x00000014, final instruction 0x00000013, and total cycles = 67.

4.4 Example Program Instructions

Instruction Word	Comment / Purpose
00F00093	RISC-V immediate operation used in the CPU bring-up sequence.
04000113	Immediate value 0x40 used for GPIO-related addressing/data setup.
00112023	Store operation using the generated register value.
08000113	Immediate value 0x80 in the test sequence.
00112023	Store operation.
0C000113	Immediate value 0xC0 in the test sequence.
00112023	Store operation.
10000113	Immediate value 0x100 in the test sequence.
00112023	Store operation.
14000113	Immediate value 0x140 in the test sequence.
00112023	Store operation.
18000113	Immediate value 0x180 in the test sequence.
01000193	Immediate value 0x10 used in a second register.
00312023	Store operation using the second register.
02000193	Immediate value 0x20.
00312223	Store operation.
00100193	Immediate value 0x1.
00312423	Store operation.
00100193	Immediate value 0x1.
00312623	Store operation.
0000006F	RISC-V jump instruction used at the end of the supplied sequence.

CHAPTER 5: RESULTS AND VERIFICATION

5.1 Verification Methodology

The project was verified using self-checking testbenches. Individual subsystem tests were used to validate functional behavior, while an integrated SoC testbench checked CPU boot, AXI4-Lite activity and peripheral/DMA/interrupt integration.

5.2 Final SoC Verification

Verification Item	Result
Reset / Idle	PASS
CPU Boot	PASS
AXI4-Lite Activity	PASS
GPIO	PASS
Timer	PASS
UART	PASS
SPI	PASS
I²C	PASS
DMA	PASS
Interrupt Controller	PASS
Overall	10 / 10 tests passed

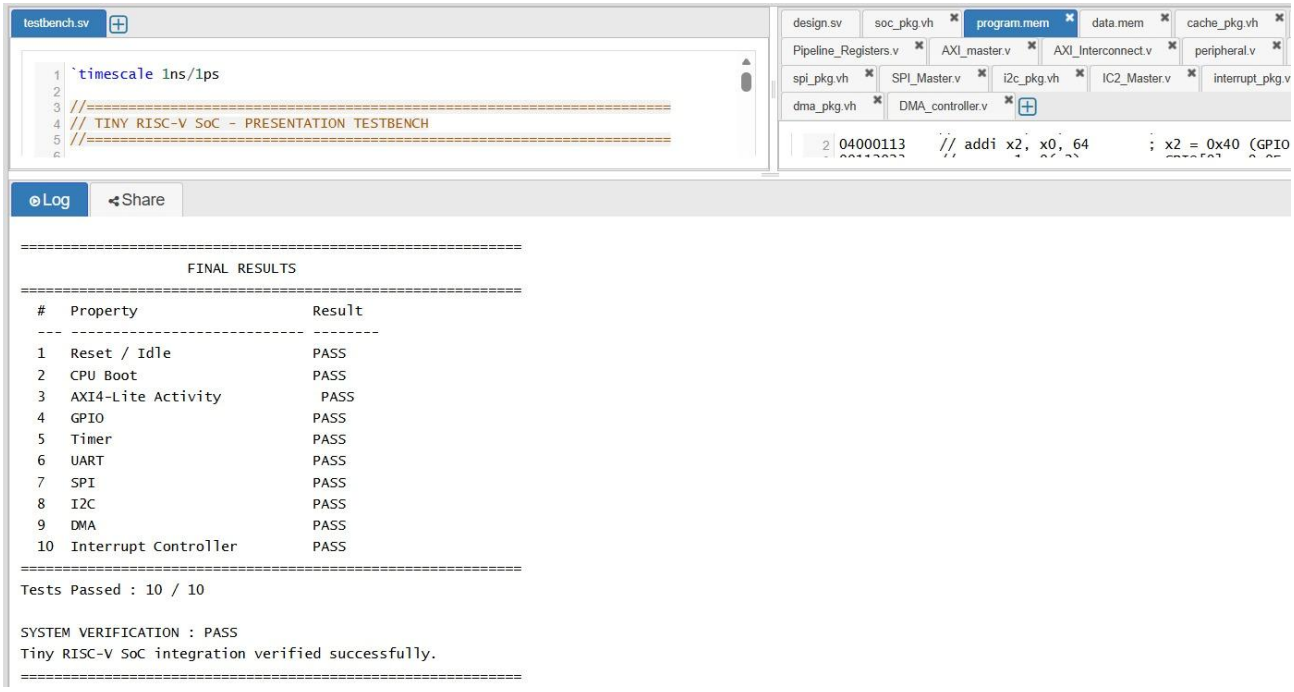


Figure 5.1: Final SoC verification report.

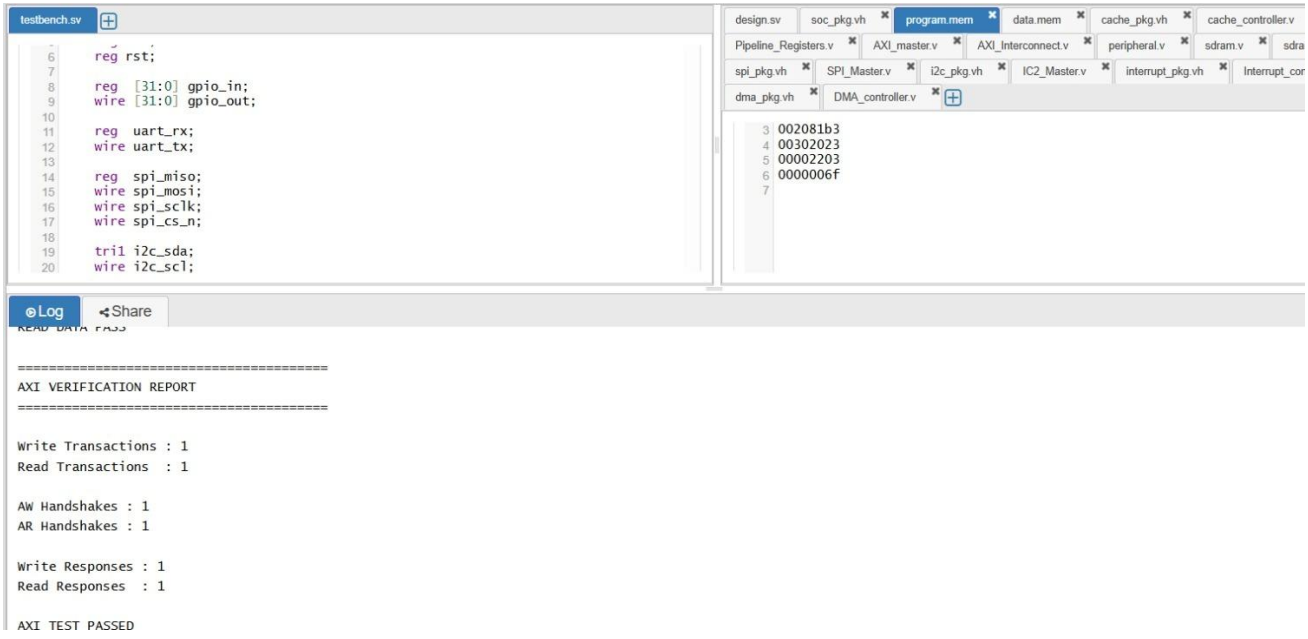
5.3 CPU Bring-up Results

Item	Observed Result
x1	0x00000005
x2	0x0000000A
x3	0x0000000F
x4	0x0000000F
Final PC	0x00000014
Final Instruction	0x00000013
Total Cycles	67
CPU Bring-up	PASS

```
Ready = 0

DMA
Busy = 0
Done = 0

Interrupt
IRQ    = 0
IRQ ID = 0

=====================================================
          Tiny SoC CPU Verification Report
=====================================================
x1 PASS : 00000005
x2 PASS : 0000000a
x3 PASS : 0000000f
x4 PASS : 0000000f

Final PC          = 00000014
Final Instruction = 00000013
Total Cycles      = 67

#############################################
#                                           #
#       CPU BRING-UP TEST PASSED            #
#                                           #
#############################################
```

5.4 AXI4-Lite Verification

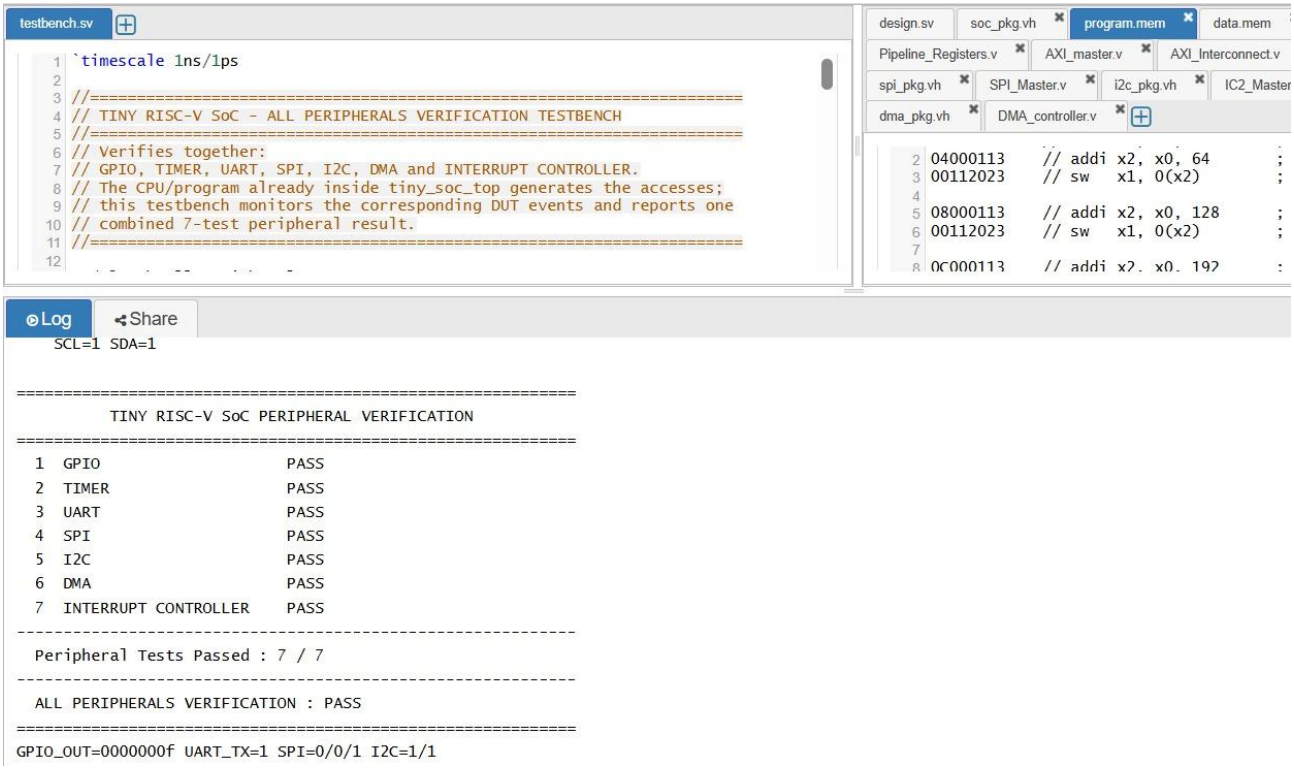
Metric	Result
Write Transactions	1
Read Transactions	1
AW Handshakes	1
AR Handshakes	1
Write Responses	1
Read Responses	1
Overall	AXI TEST PASSED



5.5 Peripheral Verification

Module	Verification Result
SDRAM	7 / 7 PASS
GPIO	7 / 7 PASS
UART	7 / 7 PASS
SPI	7 / 7 PASS
I²C	7 / 7 PASS
Interrupt Controller	7 / 7 PASS
DMA	7 / 7 PASS

The individual reports verify reset, initialization and functional transactions appropriate to each block. The DMA report additionally records four DMA reads and four DMA writes, with 7/7 tests passed.



5.6 DMA + Cache Integration

The DMA-cache integration test verifies destination data integrity after transfer. The observed report shows the tested destination words matching the expected values and records 7/7 tests passed.

testbench.sv

```
1  `timescale 1ns/1ps
2
3  `include "dma_pkg.vh"
4  `include "cache_pkg.vh"
5
6  module tb_dma_cache_integration;
7
8      //------------------------------------------------------------
9      // Clock / Reset
10     //------------------------------------------------------------
```

design.sv | soc_pkg.vh | program.mem | data.mem | cache_pkg.vh | Pipeline_Registers.v | AXI_master.v | AXI_Interconnect.v | peripheral.v | sdram.v | spi_pkg.vh | SPI_Master.v | i2c_pkg.vh | IC2_Master.v | interrupt_pkg.vh | dma_pkg.vh | DMA_controller.v

```
572        .transfer_length(transfer_length),
573
574        .mem_read_data(mem_read_data),
575
576
```

Log | Share

```
--------------------------------------------
DESTINATION VERIFICATION
--------------------------------------------
Address 200 = aaaa1111
Property 5 (Word 0) : PASS
Address 20C = dddd4444
Property 6 (Word 3) : PASS
Property 7 (Full Transfer Integrity) : PASS

============================================
Tests Passed : 7 / 7
============================================

############################################
#
#   DMA CACHE INTEGRATION TEST PASSED
#
############################################
```

5.7 System Waveform Analysis

The integrated waveform contains clock/reset, AXI address and data handshaking, GPIO write enable, peripheral write enables, DMA memory-read and memory-write activity, DMA completion, and the interrupt signal. This waveform provides evidence that the major control paths become active in the expected sequence.

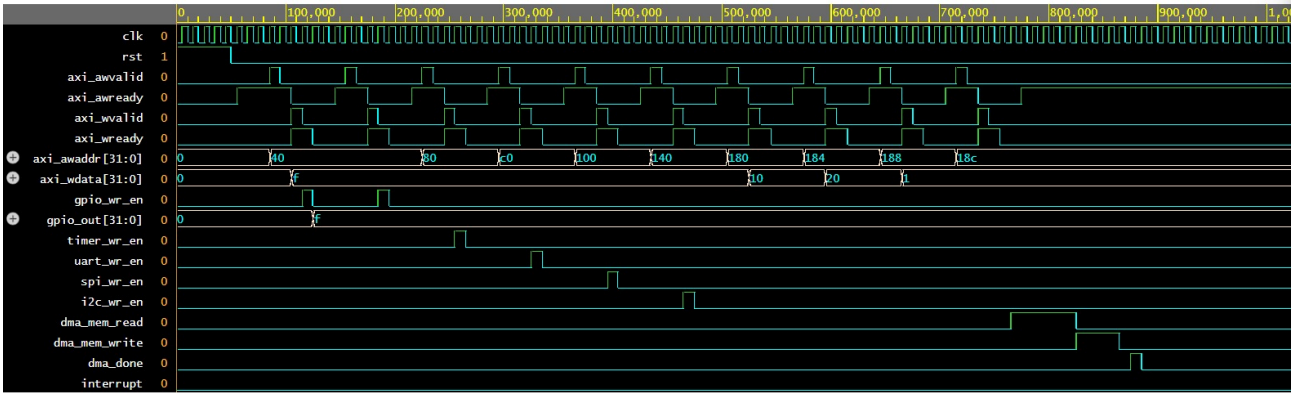


Figure 5.2: Integrated SoC waveform showing AXI, peripheral, DMA and interrupt activity.

CHAPTER 6: ADVANTAGES, LIMITATIONS AND APPLICATIONS

6.1 Advantages

- Open RISC-V based processor architecture suitable for educational and research use.
- Modular RTL structure makes subsystem-level development and verification easier.
- 5-stage pipeline provides a clear processor datapath structure.
- 2-way set-associative instruction/data cache improves memory access organization.
- AXI4-Lite provides a standard memory-mapped communication structure.
- DMA reduces CPU involvement in bulk data movement.
- Multiple standard peripherals make the SoC extensible.
- Interrupt controller provides centralized event management.
- Self-checking verification gives clear PASS/FAIL results.

6.2 Limitations

- The current evidence is primarily simulation-based; FPGA hardware results should be reported only after actual hardware validation.
- AXI4-Lite is intended for low-complexity memory-mapped control/data transactions and does not provide AXI4 burst functionality.
- The cache implementation is a compact project-oriented design and may require further optimization for high-performance systems.
- The design has limited scalability compared with large commercial SoC interconnects and cache hierarchies.
- Peripheral functionality is focused on the implemented project interfaces rather than a complete commercial peripheral ecosystem.
- Performance, area, power and timing figures require synthesis/implementation results before they can be claimed.

6.3 Applications

- Embedded RISC-V processor research.
- Educational SoC and computer architecture laboratories.
- RTL design and verification training.
- FPGA-based processor prototyping after hardware implementation.
- Peripheral and AXI4-Lite interface research.
- DMA and interrupt-controller experimentation.
- Cache and memory-system studies.
- Small embedded control systems.

CHAPTER 7: FUTURE SCOPE

- Implement and validate the complete SoC on an FPGA development board.
- Perform synthesis, timing analysis and resource-utilization analysis.
- Add AXI4/AXI4-Lite bridges or a richer interconnect if higher-throughput memory traffic is required.
- Improve cache features such as replacement policy, refill handling and performance monitoring.
- Add more memory types and robust memory-controller features.
- Expand DMA to support more channels and additional transfer modes.
- Add timer, UART, SPI and I²C interrupt sources with richer register interfaces.
- Introduce formal verification or UVM-based verification for protocol and subsystem checking.
- Measure area, maximum clock frequency and dynamic power after synthesis.
- Integrate the SoC into a larger FPGA/ASIC-oriented system.

CHAPTER 8: CONCLUSION

The Tiny RISC-V SoC was designed as a modular processor-based system integrating a 5-stage RISC-V RV32IM CPU, cache subsystem, AXI4-Lite interconnect, SDRAM controller, DMA controller, interrupt controller and multiple peripherals.

The verification evidence demonstrates successful CPU bring-up, AXI4-Lite read/write handshaking, memory operation, peripheral functionality, DMA operation, interrupt-controller behavior and DMA-cache integration. The final integrated verification report records 10/10 tests passed.

The project therefore provides a practical foundation for understanding how processor, cache, memory, bus, peripheral, DMA and interrupt blocks are combined into a small System-on-Chip. Future FPGA validation, synthesis analysis and advanced verification can extend the design toward a more complete implementation.

CHAPTER 9: REFERENCES

18. RISC-V International, The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA.
19. Arm Limited, AMBA AXI and ACE Protocol Specification.
20. Samir Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, Pearson.
21. David A. Patterson and John L. Hennessy, Computer Organization and Design: RISC-V Edition.
22. Icarus Verilog documentation and simulation environment.
23. EPWave waveform viewer documentation.
24. Project RTL source files, simulation testbenches, program/data memory files and verification logs.